Analytics Engineering: Session 10

Advanced Materializations: Incremental & Snapshots

Agenda

- Why Incremental Models?
- Incremental Strategies
- The `is_incremental()` Macro
- Handling Edge Cases
- Snapshots (SCD Type 2)
- Incremental vs Snapshots

Why Incremental Models?

The problem with rebuilding large tables every time:

- Full refreshes become slow with large tables
- Resource intensive for frequent updates
- Costly in cloud data warehouses

The Solution

- Process only new/changed data
- Maintain historical context
- Reduce compute costs

Incremental Models

For large tables, rebuilding from scratch every time is too slow/expensive.

Incremental: Only process new or changed data.

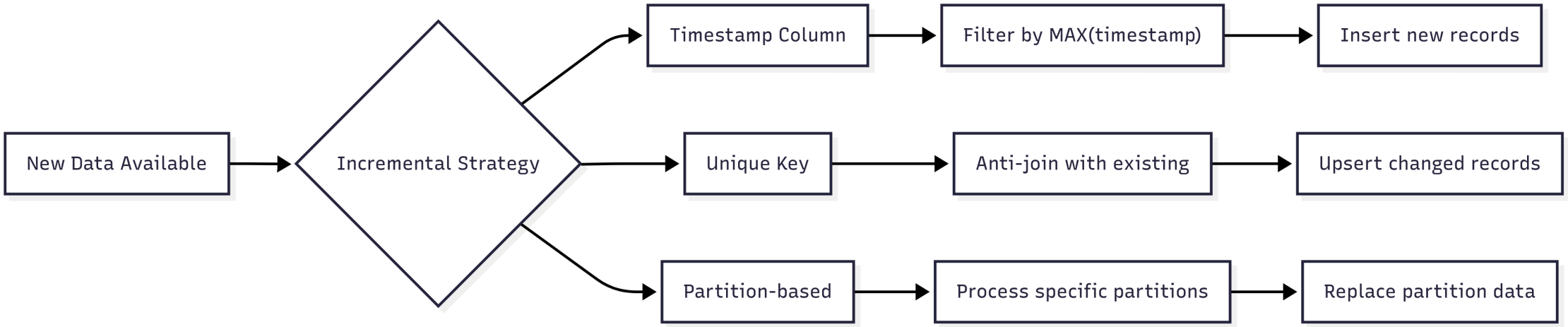
```
{{
    config(
        materialized='incremental',
        unique_key='order_id'
    )
}}

select * from {{ ref('stg_orders') }}
{% if is_incremental() %}
  where order_date > (select max(order_date) from {{ this }})
{% endif %}
```

Incremental Strategies

How dbt updates the destination table.

- **append**: Just add new rows (duplicates possible).
- **merge**: Update existing rows, insert new ones (requires `unique_key`).
- **delete+insert**: Delete rows that match, then insert.

Incremental Model Patterns



Timestamp-based Incremental

```
{{ config(materialized='incremental') }}

select *
from {{ ref('stg_orders') }}
{% if is_incremental() %}
  where order_date > (select max(order_date) from {{ this }})
{% endif %}
```

Unique Key-based Incremental

```
{{ config(
  materialized='incremental',
  unique_key='order_id'
) }}

select *
from {{ ref('stg_orders') }}
{% if is_incremental() %}
  where order_id not in (select order_id from {{ this }})
{% endif %}
```

Handling Edge Cases

Late-arriving Data

```
-- lookback window for late data
{% if is_incremental() %}
  where order_date > dateadd(day, -7, (select max(order_date) from {{ this }}))
{% endif %}
```

Schema Changes

```
{{ config(
    materialized = 'incremental',
    unique_key = 'id',
    on_schema_change = 'sync_all_columns'
) }}
```


- `ignore` - Do nothing, keep existing schema

Full Refresh

When you need to rebuild an incremental model from scratch:

```
dbt run -s mart_orders --full-refresh
```

When to use:

- Schema changes that require a full rebuild
- Data quality issues that affected historical data
- Logic changes that need to be applied retroactively

Snapshots in dbt

Snapshots capture and store historical versions of records over time.

When to Use Snapshots

- Slowly changing dimensions (Type 2 SCD)
- Audit trails for compliance
- Point-in-time analysis

Basic Snapshot

```
{% snapshot customer_snapshot %}

{{
  config(
    target_schema='snapshots',
    unique_key='customer_id',
    strategy='timestamp',
    updated_at='updated_at'
  )
}}

select * from {{ source('raw', 'customers') }}

{% endsnapshot %}
```


dbt adds columns: `dbt_valid_from` `dbt_valid_to` `dbt_scd_id` `dbt_updated_at`

Snapshot Strategies

Timestamp: Compare `updated_at` column to detect changes.

```
strategy='timestamp',
updated_at='updated_at'
```

Check: Compare specific columns to detect changes.

```
strategy='check',
check_cols=['email', 'address', 'phone']
```

Snapshots vs Incremental Models

Aspect	Incremental Models	Snapshots
Use Case	Fact tables, events	Dimension tables, history
Performance	Fast, targeted updates	Slower, full comparisons
Storage	Current state only	Historical versions
Complexity	Higher (custom logic)	Lower (dbt handles)
Cost	Lower (less processing)	Higher (version storage)

Best Practices

Monitoring	Testing	Maintenance
Track incremental processing time	Test incremental logic separately	Regular cleanup of old snapshots
Monitor data freshness	Validate data consistency	Monitor storage costs
Alert on processing failures	Check for duplicates or gaps	Plan for schema evolution

What have we learned in this session

- Convert a table model to incremental with different strategies
- Use `is_incremental()` and `{{ this }}` for incremental logic
- Handle edge cases (late data, schema changes, full refresh)
- Create snapshots for SCD Type 2 historical tracking
- Choose between timestamp and check strategies for snapshots

Next Session: Model Governance: Contracts, Versions & Access.